IERG4320/IEMS5726: Data Science in Practice

Lecture 10: Case Studies (II) Natural Language Problems

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

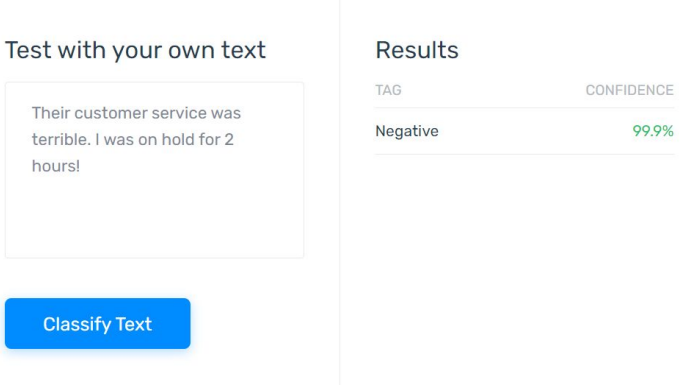
Outline

1. Introduction to Natural Language Processing
2. Semantic Analysis
   - Neural Bag of Words (NBoW) Approach
   - Long Short-term Memory (LSTM) Model Approach
   - Bidirectional Encoder Representations from Transformers (BERT) Model Approach
   - Transform Model Approach
3. Text Generation
   - Markov Chain Approach
   - Generative Pre-trained Transformers (GPT) Model Approach
   - Large Language Model Meta AI (LLaMA) Model Approach

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

Applications for Natural Language Processing

1. Sentiment Analysis
2. Text Classification
3. Chatbots and Virtual Assistants
4. Text Extraction
5. Machine Translation
6. Text Summarization
7. Auto-Correct
8. Intent Classification
9. Speech Recognition
10. Report Generation
11. Image Captioning



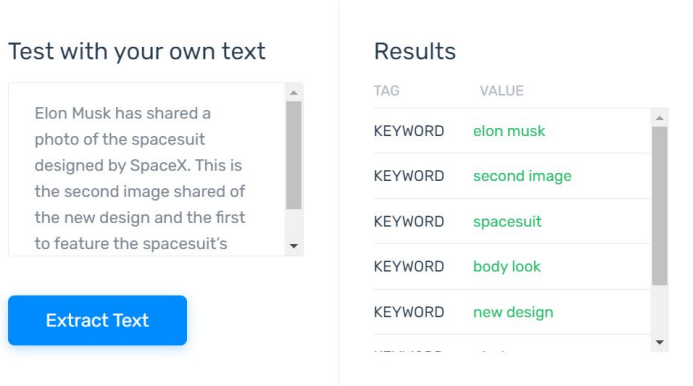


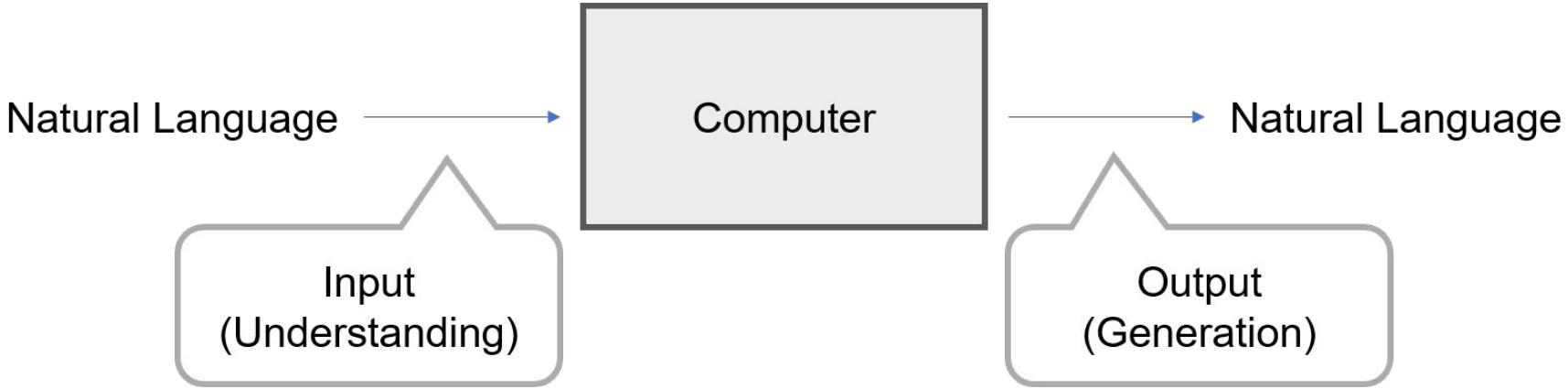
Image credit: https://monkeylearn.com/blog/natural-language-processing-applications/

Why Natural Language Processing Difficult?

1. **High-Dimensional**: Natural language data, when converted into numerical features (word embeddings, TF-iDF), creates a very high-dimensional feature space. There can be tens or hundreds of thousands of unique words .
2. **Sparse**: For any single document, only a tiny fraction of the total vocabulary is used. This results in feature vectors that are mostly zeros, which is known as sparsity.
3. **Sequential and Context-Dependent**: The meaning of text relies on the order of words and context.
4. **Noisy** and **Unstructured**: Text often contains typos, slang, abbreviations, emojis and grammatical errors. Unseen features (new words, phrases) are created daily.
5. **Language Specific Features**: Things can become very different across different languages
6. … etc

Natural Language Processing (NLP)

- NLP involves making the computer **understand** natural language input and **generate** natural language output
- It is a field that involves computer science, linguistics, artificial intelligence, human-computer interface, etc.



Large Language Model (LLM)

- LLM is a language model trained with self-supervised machine learning on a vast amount of text, designed for natural language processing tasks, especially language generation.
- LLMs consist of billions to trillions of parameters and operate as general-purpose sequence models, **generating**, **summarizing**, **translating** and **reasoning** over text.
- LLMs evolved from earlier statistical and recurrent neural network approaches to language modeling. The transformer architecture, introduced in 2017, enabled models like **GPT**, **BERT**, **LLaMa**, **DeepSeek**, **Gemini**, … etc.

A Classic NLP Problem: Fake News Detection

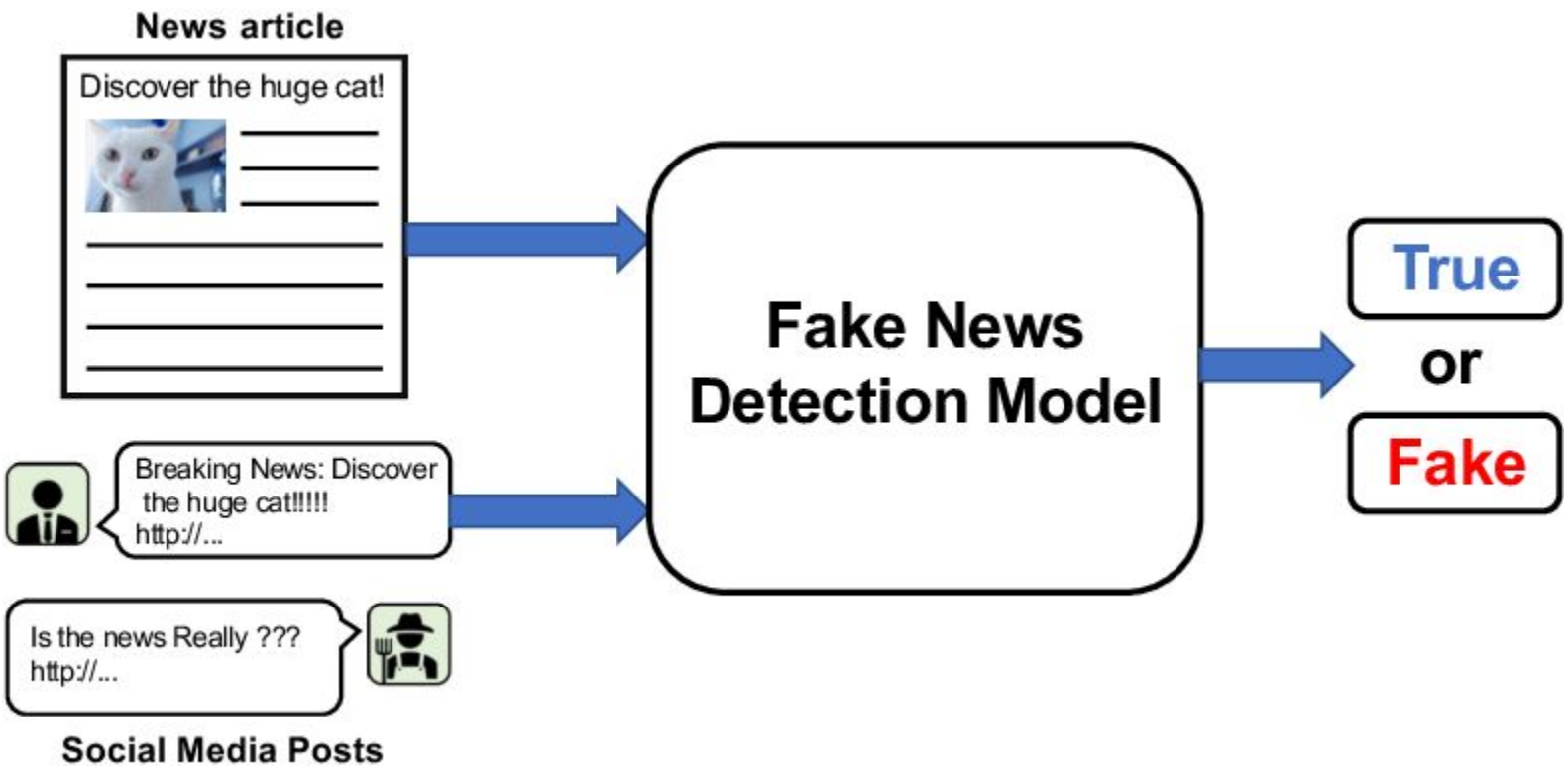


Image credit: Semantic Scholar

Fake News Detection: Classic Approach

1. Data Collection and Labeling
2. Text Preprocessing
3. Generating Word Embeddings
4. Creating a Document Representation
5. Building and Training a Classifier, e.g. Logistic regressor, support vector regressor, random forest classifier, XGBoost classifier, … etc
6. Evaluating the Prediction

Fake News Detection: LLM Approach

1. Data Collection and Labeling
2. Text Preprocessing and Tokenization
3. Model Selection and Input Representation
4. The Transformer's Core Processing (Forward Pass)
5. Adding a Classification Head
6. Fine-Tuning the Entire Model
7. Evaluation and Inference

Why the Transformer Approach is Better?

1. **Information is King**: Transformer models can understand antonyms, but classic machine learning approaches fail.
2. **Long-Range Dependencies**: Transformer models can connect a claim made in the first paragraph to a statement in some later paragraphs.
3. **Handles Nuance and Style**: It can detect the subtle, emotionally charged and conspiratorial language patterns that are hallmarks of fake news, even if the individual words are neutral.
4. **Transfer Learning Power**: Starting from a model pre-trained on a vast corpus gives it a strong foundational understanding of truth and logic, which we then fine-tune to detect deviations from it.

Semantic Analysis on Movie Review

Image credit: Google Image

Problem Definition - Semantic Analysis

- A process of understanding natural language (text) by extracting insightful information such as **context**, **emotions**, and **sentiments** from unstructured data
- Common goals:

1. To **drive business intelligence** and **automation**, e.g. Netflix, YouTube, Spotify use semantic analysis to understand the theme, genre and mood of the content.
2. To **set up the foundations for AI applications**, e.g. If you say, “affordable fast food near me that are open now”, the machine will understand:
    - “affordable” means price range, cheap, budget-friendly
    - “fast food” is the cuisine type, entity
    - “near me” refers to user’s current location
    - “open now” captures both current time and operating hours.

Main Types of Semantic Analysis

1. **Context**: The study of the word and word meaning.
   - E.g. Translating one language to another language
2. **Emotions**: The study of the feeling of the writer, i.e. happy, sad, fear and calm.
   - E.g. Analyzing the diary (or weekly diary) of the students.
3. **Sentiments**: The study of the impression of the writer, i.e. positive, neutral and negative.
   - E.g. Analyzing the customers' product review

Non-Machine Learning Approaches

1. Data processing.
2. Defining features, parameters, and characteristics of processed data
3. Data representation
4. Defining grammar for data analysis
5. Assessing semantic layers of processed data
6. Performing semantic analysis based on the linguistic formalism

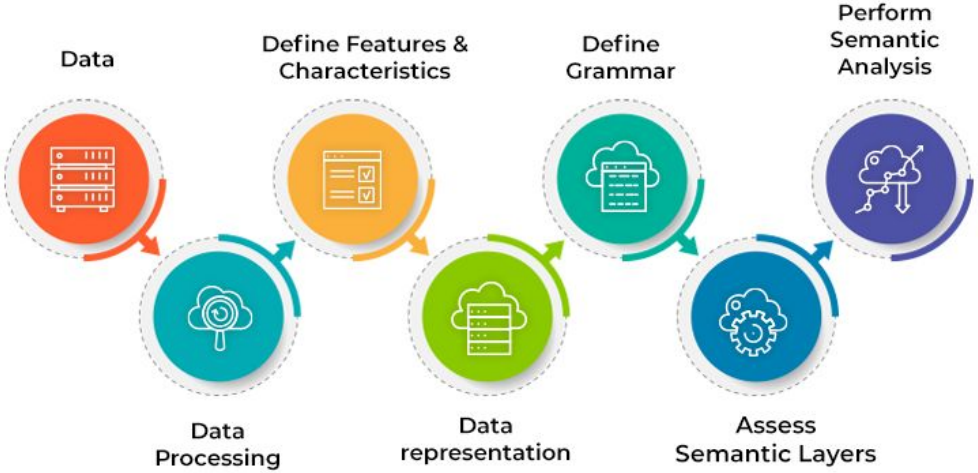


Image credit: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-semantic-analysis/

Non-Machine Learning Approaches

1. **Hyponyms**: This refers to a specific lexical entity having a relationship with a more generic verbal entity called hypernym. For example, **red, blue, and green are all hyponyms of color**, their hypernym.
2. **Meronomy**: Refers to the arrangement of words and text that denote a minor component of something. For example, **mango is a meronomy of a mango tree**.
3. **Polysemy**: It refers to a word having more than one meaning. However, it is represented under one entry. For example, the term **'dish' is a noun**. In the sentence, **'arrange the dishes on the shelf,'** the word **dishes refers to a kind of plate**.
4. **Synonyms**: This refers to **similar-meaning** words. For example, abstract (noun) has a synonyms summary–synopsis.
5. **Antonyms**: This refers to words with **opposite meanings**. For example, cold has the antonyms warm and hot.
6. **Homonyms**: This refers to words with the same spelling and pronunciation, but reveal a different meaning altogether. For example, **bark (tree) and bark (dog)**.

A Classic Machine Learning Approach

1. Collect the movie review dataset, and separate into training set, testing set.
   - Training size: 25,000; Testing size: 25,000
2. Extra the TF-iDF from the data
   - Training size: #samples: 25,000, #features: 5000
3. Employ traditional classifier for semantic prediction (Positive or Negative)
   - Naive Bayes classifier

Dataset: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=p78EUklL1rSY

A Classic Machine Learning Approach

```
Accuracy: 0.8391

Classification Report:
              precision    recall  f1-score   support

    Negative       0.83      0.86      0.84     12500
    Positive       0.85      0.82      0.84     12500

    accuracy                           0.84     25000
   macro avg       0.84      0.84      0.84     25000
weighted avg       0.84      0.84      0.84     25000
```

Discussion: A Classic Machine Learning Approach

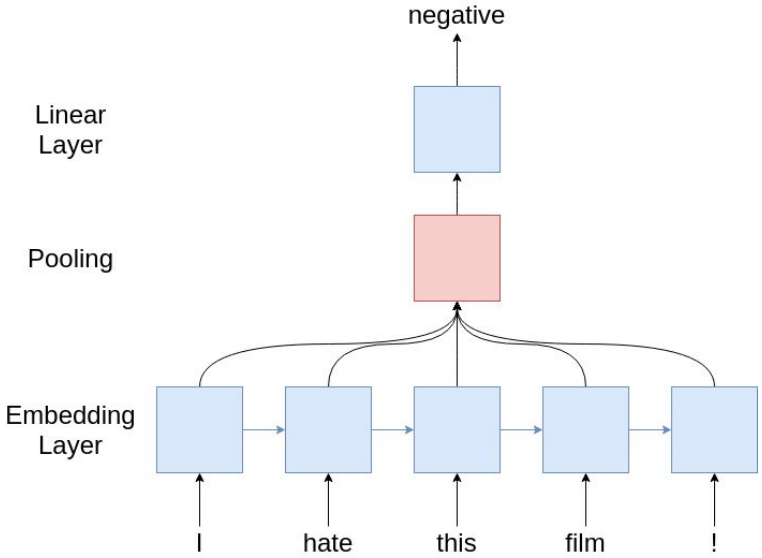
1. Did this approach solve all the NLP problems?
   - Polysemy: It refers to a word having more than one meaning.
   - Homonyms: This refers to words with the same spelling and pronunciation, but reveal a different meaning altogether.
2. TF-iDF cannot capture position in text.
3. Naive Bayes classifier is a beginner machine learning model.
4. Is F-measure 0.83 satisfactory?

Neural Bag-of-Words (NBoW) Model

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=8KRIjZPk51Y2

Introduction to Neural Bag-of-Words Model

- The Neural bag-of-Words (NBoW) model performs classification with an average of the input word vectors and achieves an impressive performance.
- While the NBOW model learns word vectors targeted for the classification task, it does not explicitly model which words are important for given task.
- **Traditional BoW**: Counts word frequencies, loses word order and semantics
- **NBoW**: Uses word embeddings + neural network to capture semantic meaning while maintaining BoW simplicity



Steps in Preparing the NBOW Model

1. Text Input and Tokenization

```
Input: "The movie was absolutely fantastic and engaging"
→ Tokenization: ["the", "movie", "was", "absolutely", "fantastic", "and",
"engaging"]
```

2. Word to Embedding Lookup: Each word is converted to a dense vector using an embedding layer

```
"the"    → [0.1, 0.2, -0.1, ..., 0.3]  (d-dimensional vector)
"movie"  → [0.4, -0.2, 0.5, ..., -0.1]
"was"    → [0.0, 0.1, 0.2, ..., 0.4]
...
"engaging" → [0.7, 0.3, -0.4, ..., 0.2]
```

Steps in Preparing the NBOW Model

3. Bag-of-Words Aggregation: All word embeddings are combined using a pooling operation (typically **average** or **sum pooling**)

```
Document Vector = (embed("the") + embed("movie") + ... + embed("engaging")) / 7
```

4. Classification: The aggregated document vector is fed through a classifier (usually a feedforward neural network) to make predictions.

Example: NBoW Model Architecture

```
Input Text
     ↓
Tokenization → ["word1", "word2", ..., "wordN"]
     ↓
Embedding Layer → [vec1, vec2, ..., vecN]  (each vec ∈ R^d)
     ↓
Pooling Layer → Average/Sum/Max Pooling
     ↓
Document Vector (fixed-size representation)
     ↓
Hidden Layers (Optional)
     ↓
Output Layer (Softmax)
     ↓
Classification
```

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=8KRIjZPk51Y2

Example: NBoW Model Architecture

```python
class NeuralBagOfWords(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx):
        super(NeuralBagOfWords, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.fc1 = nn.Linear(embed_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(0.3)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        embedded = self.embedding(x)
        bag_of_words = torch.mean(embedded, dim=1)
        out = self.dropout(self.relu(self.fc1(bag_of_words)))
        out = self.dropout(self.relu(self.fc2(out)))
        out = self.sigmoid(self.fc3(out))
        return out.squeeze(-1)

```

```
Using device: cpu
Loading IMDB dataset...
Building simple vocabulary...
Vocabulary size: 25000
PAD_IDX: 0, UNK_IDX: 1
Model parameters: 2,529,569
Padding index: 0 (valid: 0 <= 0 < 25000)

Starting training...

Epoch 1/5
Batch 0, Loss: 0.6910
Batch 100, Loss: 0.6236
Batch 200, Loss: 0.5380
Batch 300, Loss: 0.4524
Train Loss: 0.5534, Train Acc: 0.7074
Test Acc: 0.8159
New best model saved!

Epoch 2/5
Batch 0, Loss: 0.5441
Batch 100, Loss: 0.1996
Batch 200, Loss: 0.3931
Batch 300, Loss: 0.3221
Train Loss: 0.3323, Train Acc: 0.8594
Test Acc: 0.8502
New best model saved!
```

```
Epoch 3/5
Batch 0, Loss: 0.3442
Batch 100, Loss: 0.2398
Batch 200, Loss: 0.3016
Batch 300, Loss: 0.3573
Train Loss: 0.2411, Train Acc: 0.9066
Test Acc: 0.8603
New best model saved!

Epoch 4/5
Batch 0, Loss: 0.2999
Batch 100, Loss: 0.1806
Batch 200, Loss: 0.1450
Batch 300, Loss: 0.1067
Train Loss: 0.1803, Train Acc: 0.9341
Test Acc: 0.8624
New best model saved!

Epoch 5/5
Batch 0, Loss: 0.2003
Batch 100, Loss: 0.0742
Batch 200, Loss: 0.1007
Batch 300, Loss: 0.1539
Train Loss: 0.1355, Train Acc: 0.9541
Test Acc: 0.8639
New best model saved!
Best Test Accuracy: 0.8639
Final Test Accuracy (best model): 0.8639
```

Advantages of NBoW Model

1. **Simple and Fast**: Much simpler than RNN, LSTM, Transformers
2. **Fixed-size Output**: Always produces same-sized document vector regardless of input length
3. **Parallelizable**: No sequential processing needed
4. **Good Baseline**: Often performs surprisingly well for classification tasks
5. **Interpretable**: Can analyze which words contribute most to decisions

Limitations

1. **Word Order Ignored**: “a dog bites a man” vs “a man bites dog” treated identically
2. **No Context Awareness**: Each word is processed independently
3. **Limited Semantic Understanding**: Cannot capture complex linguistic phenomena
4. **Vocabulary Dependent**: Performance depends heavily on vocabulary coverage

Long Short-Term Memory (LSTM) Models Again

1. Sequential Text Input

```
Input: "The movie was absolutely fantastic"
→ Tokenization: ["The", "movie", "was", "absolutely", "fantastic"]
→ Sequential Processing:
   Time Step 1: "The"
   Time Step 2: "movie"
   Time Step 3: "was"
   Time Step 4: "absolutely"
   Time Step 5: "fantastic"
```

2. Word Embedding at Each Step: Each word is converted to a dense vector

```
"The" → [0.1, 0.2, -0.1, 0.3]
"movie" → [0.4, -0.2, 0.5, -0.1]
"was" → [0.0, 0.1, 0.2, 0.4]
...
```

Long Short-Term Memory (LSTM) Models Again

3. Sequential Processing with Memory: At each time step, the LSTM
    - Takes current word embedding + previous hidden state
    - Uses gates to decide what to remember/forget
    - Updates its internal memory
    - Produces output for current step

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=AApYTLPfDC0d

Example: LSTM Model Architecture

```python
class NeuralBagOfWords(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx):
        super(NeuralBagOfWords, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.fc1 = nn.Linear(embed_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(0.3)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        embedded = self.embedding(x)
        bag_of_words = torch.mean(embedded, dim=1)
        out = self.dropout(self.relu(self.fc1(bag_of_words)))
        out = self.dropout(self.relu(self.fc2(out)))
        out = self.sigmoid(self.fc3(out))
        return out.squeeze(-1)

```

Example: LSTM Training Results

```
Epoch 1/5, Loss: 0.6934
Epoch 2/5, Loss: 0.6908
Epoch 3/5, Loss: 0.6849
Epoch 4/5, Loss: 0.7117
Epoch 5/5, Loss: 0.6909

Accuracy: 0.5315

Classification Report:
              precision    recall  f1-score   support

    Negative       0.53      0.53      0.53     12500
    Positive       0.53      0.53      0.53     12500

    accuracy                           0.53     25000
   macro avg       0.53      0.53      0.53     25000
weighted avg       0.53      0.53      0.53     25000
```

Longer training process is needed.

```
Example Predictions:

Review: I love sci-fi and am willing to put up with a lot. Sci-fi movies/TV are usually underfunded,
under-a...
True Label: Negative
Predicted Label: Negative

Review: Worth the entertainment value of a rental, especially if you like action movies. This one features
t...
True Label: Negative
Predicted Label: Positive

Review: its a totally average film with a few semi-alright action sequences that make the plot seem a
little...
True Label: Negative
Predicted Label: Positive

Review: STAR RATING: ***** Saturday Night **** Friday Night *** Friday Morning ** Sunday Night * Monday
Morn...
True Label: Negative
Predicted Label: Positive

Review: First off let me say, If you haven't enjoyed a Van Damme movie since bloodsport, you probably will
n...
True Label: Negative
Predicted Label: Positive
```

Summary for LSTM Model

1. **Solving the vanishing gradient problem** through gating mechanisms
2. Enabling **long-range dependency** learning in text
   - But failed for very long sequences
3. Providing **contextual understanding** through sequential processing
4. Handling **complex linguistic structures** that simpler models miss
5. However, sequential nature **limits parallel processing**

Bidirectional Encoder Representations from Transformers (BERT) Model

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=GOIE6xHIVWw6

Bidirectional Encoder Representations from Transformers (BERT)

- BERT is a transformer-based machine learning technique for natural language processing (NLP) pre-training developed by Google in 2018.
- **Bidirectional Context**: Understands words in full context
- **Transformer Architecture**: Uses self-attention mechanism
- **Pre-training** and **Fine-tuning**: Learns general language understanding then adapts to specific tasks
- **State-of-the-Art Performance**: Achieved new benchmarks on 11 NLP tasks
  - GLUE Benchmark Tasks (9 tasks)
  - SQuAD Tasks (2 tasks)

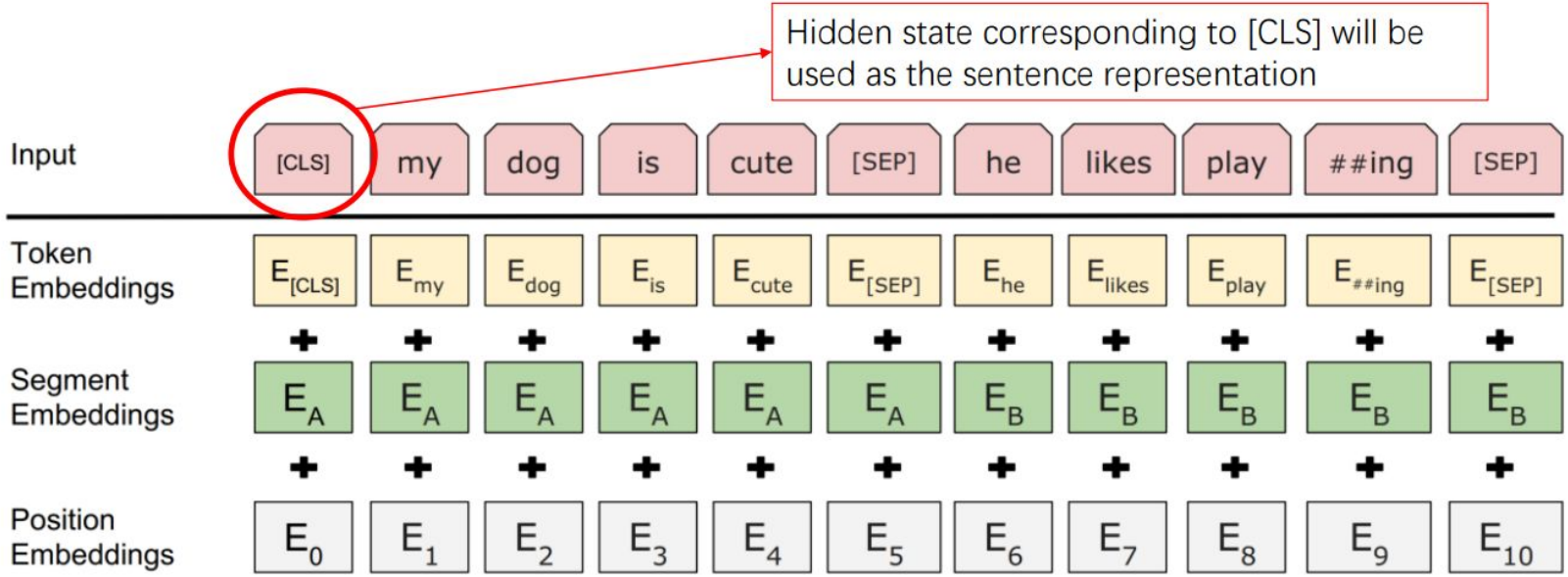
BERT Models

1. Input Representation: combines three types of embeddings
   - `input_rep = token_embedding + segment_embedding + position_embedding`

```
Input: "The cat sat on the mat"
→ Tokenization: ["[CLS]", "the", "cat", "sat", "on", "the", "mat", "[SEP]"]
```

2. Multi-layer Encoding: Each token flows through multiple transformer layers simultaneously, with each word attending to all other words in the sequence.
   - Multi-Head Self-Attention
   - Feed-Forward Neural Network
   - Residual Connections and Layer Normalization
3. Contextualized Output: Every input token gets a contextualized representation that incorporates information from the entire sequence.

BERT Input Representation

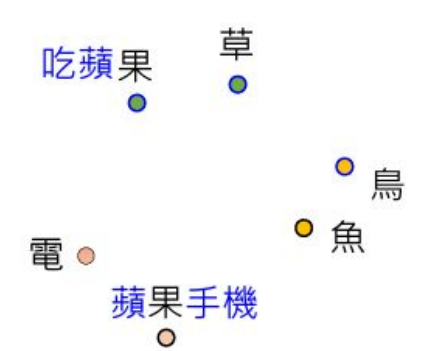


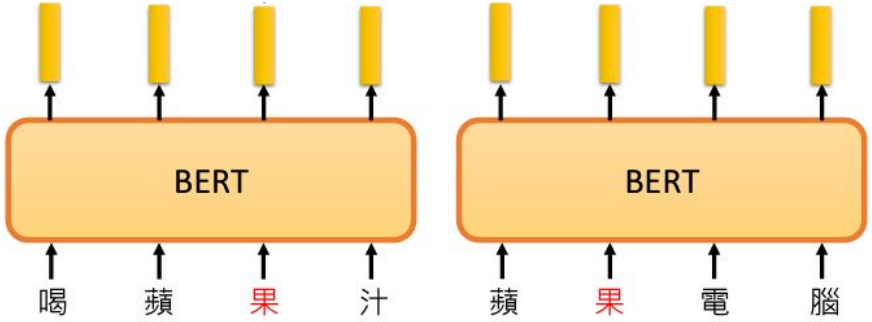
- **Token Embeddings**: WordPiece embedding (Wu et al., 2016)
- **Segment Embeddings**: randomly initialized and learned; single sentence input only adds E_A
- **Position embeddings**: randomly initialized and learned

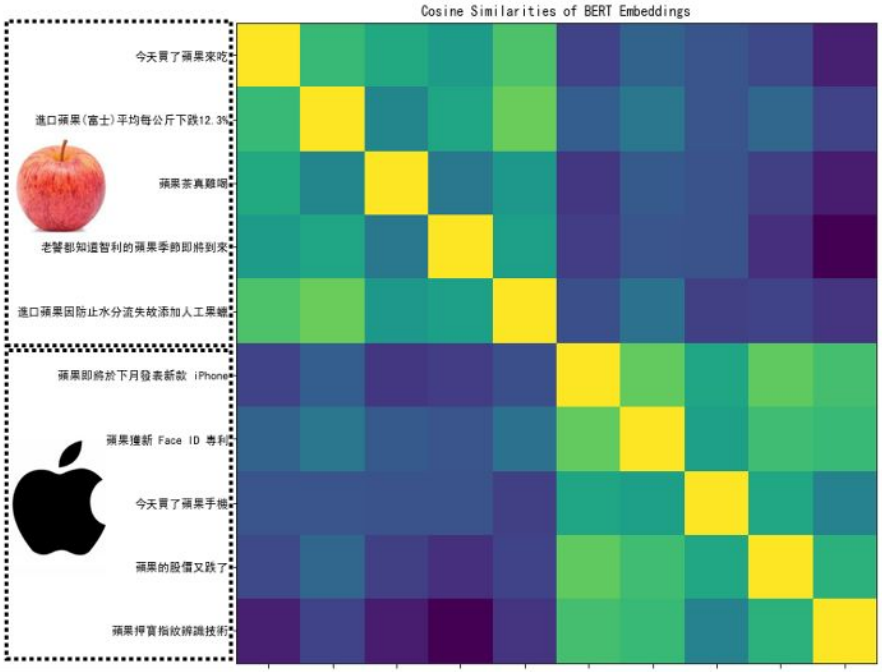
Image credit: https://cgi.csc.liv.ac.uk/~hang/ppt/BERT%20and%20Transformer.pdf

BERT Embeddings

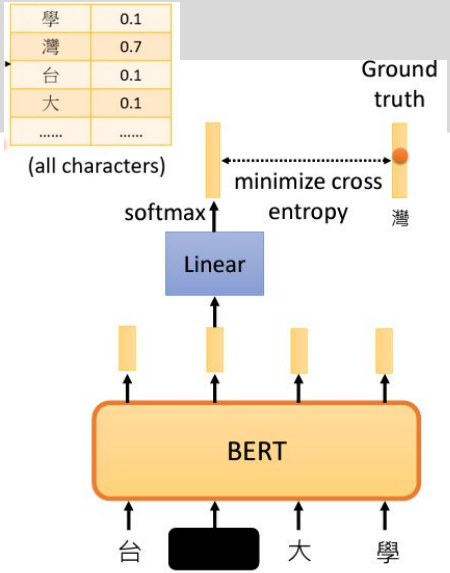
- The tokens with similar meaning have similar embedding.
- Context of the token is considered in the embedding.
- Cosine similarity is computed.







Two Pre-training Strategies for BERT

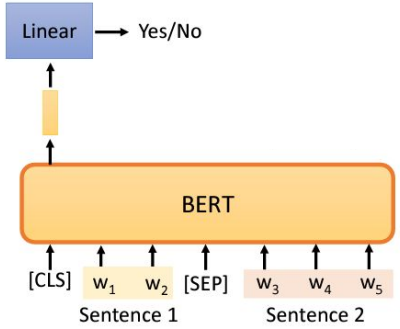


1. Language Modelling / Masking Input

- 15% of tokens are randomly masked
- Model must predict original vocabulary id of masked word
- Enables deep bidirectional representations

2. Next Sentence Prediction

- Given two sentences, predict if second follows first
- Helps understand sentence relationships



Different Variations of BERT Models

1. **ALBERT**: A lighter version of BERT model with lesser training time
2. **RoBERTa**: Facebook AI's improvement over BERT, with different pre-training steps
3. ELECTRA: New task called "replaced token detection task"
4. SpanBERT: Predicting a "span" of text
5. **DistilBERT**: A compact and faster version of BERT
    - DistilBERT Mini is fine-tuned as the demonstration
6. TinyBERT: Similar to DistilBERT

Example: DistilBERT Mini Tokenizer

- BERT model and Tokenizer work together

```
tokenizer = DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')
def tokenize_function(examples):
    return tokenizer(
        examples['text'],
        truncation=True,
        padding='max_length',
        max_length=config['max_length']
    )

# Tokenize datasets
print("Tokenizing datasets...")
train_dataset = train_dataset.map(tokenize_function, batched=True,
remove_columns=['text'])
test_dataset = test_dataset.map(tokenize_function, batched=True,
remove_columns=['text'])
```

Example: DistilBERT Mini Configuration

```python
# DistilBERT Mini Configuration
config = {
    'vocab_size': 30522,
    'dim': 256,
    'n_heads': 4,
    'n_layers': 4,
    'intermediate_dim': 1024,
    'dropout': 0.1,
    'max_length': 256,
    'num_labels': 2
}
```

Code: https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=GOIE6xHIVWw6

Example: DistilBERT Mini Model Class

```python
# DistilBERT Mini with correct masking
class DistilBertMini(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.embeddings = nn.Embedding(config['vocab_size'], config['dim'])

        # Transformer layers with proper masking support
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=config['dim'],
            nhead=config['n_heads'],
            dim_feedforward=config['intermediate_dim'],
            dropout=config['dropout'],
            batch_first=True,
            norm_first=True  # Pre-norm like DistilBERT
        )
        self.transformer = nn.TransformerEncoder(encoder_layer,
            num_layers=config['n_layers'])

```

Example: DistilBERT Mini Model Class

```
88          self.layer_norm = nn.LayerNorm(config['dim'])
89          self.dropout = nn.Dropout(config['dropout'])
90          self.classifier = nn.Linear(config['dim'], config['num_labels'])
91
92      def forward(self, input_ids, attention_mask):
93          batch_size, seq_len = input_ids.shape
94          # Embeddings
95          x = self.embeddings(input_ids)  # (batch, seq_len, dim)
96          # 2D key_padding_mask (batch_size, seq_len)
97          key_padding_mask = ~attention_mask.bool()  # True = pad, False = valid
98          # Transformer forward
99          x = self.transformer(x, src_key_padding_mask=key_padding_mask)
00          # Pooling [CLS]-style (mean pooling over non-padded tokens)
01          # More robust than just first token
02          masked_x = x * attention_mask.unsqueeze(-1)
03          pooled_output = masked_x.sum(dim=1) / attention_mask.sum(dim=1, keepdim=True)
04          # Final layers
05          pooled_output = self.layer_norm(pooled_output)
06          pooled_output = self.dropout(pooled_output)
07          logits = self.classifier(pooled_output)
08          return logits
```

```
DistilBERT Mini parameters: 10,973,698
Size vs DistilBERT: 16.6%

Starting DistilBERT Mini training...

Epoch 1/5
Batch 0, Loss: 0.8097
Batch 200, Loss: 0.5204
Batch 400, Loss: 0.4246
Batch 600, Loss: 0.4818
Train Loss: 0.5135, Train Acc: 0.7381
Test Acc: 0.8210
New best model saved!

Epoch 2/5
Batch 0, Loss: 0.3129
Batch 200, Loss: 0.4439
Batch 400, Loss: 0.2961
Batch 600, Loss: 0.3686
Train Loss: 0.3769, Train Acc: 0.8335
Test Acc: 0.8299
New best model saved!
```

```
Epoch 3/5
Batch 0, Loss: 0.3193
Batch 200, Loss: 0.2343
Batch 400, Loss: 0.2809
Batch 600, Loss: 0.2381
Train Loss: 0.3218, Train Acc: 0.8633
Test Acc: 0.8368
New best model saved!

Epoch 4/5
Batch 0, Loss: 0.1238
Batch 200, Loss: 0.2399
Batch 400, Loss: 0.3759
Batch 600, Loss: 0.1791
Train Loss: 0.2832, Train Acc: 0.8839
Test Acc: 0.8371
New best model saved!

Epoch 5/5
Batch 0, Loss: 0.1365
Batch 200, Loss: 0.3392
Batch 400, Loss: 0.2184
Batch 600, Loss: 0.0916
Train Loss: 0.2502, Train Acc: 0.9021
Test Acc: 0.8364

Best Test Accuracy: 0.8371
Final Test Accuracy: 0.8371
```

Applications for BERT

1. Semantic similarity between pairs of sentences
2. Text classification or sentence classification
3. Question answering task with paragraph
4. Text summarization
5. Named entity recognition

Reference: https://github.com/maknotavailable/pytorch-pretrained-BERT

Summary for BERT Model

1. Providing **deep bidirectional context** through transformer architecture
2. Enabling **transfer learning** via pre-training on massive corpora
3. Handling **complex linguistic phenomena** like coreference, polysemy and long-range dependencies
4. Achieving state-of-the-art results across diverse semantic tasks
5. However, the input (number of words) is **limited to the window size** and **number of available tokens**
6. The fine-tuning of the model is still expensive in terms of computational resources

Transformer Model

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=9gqrfGY6crr-

Transformer Model (Revisit)

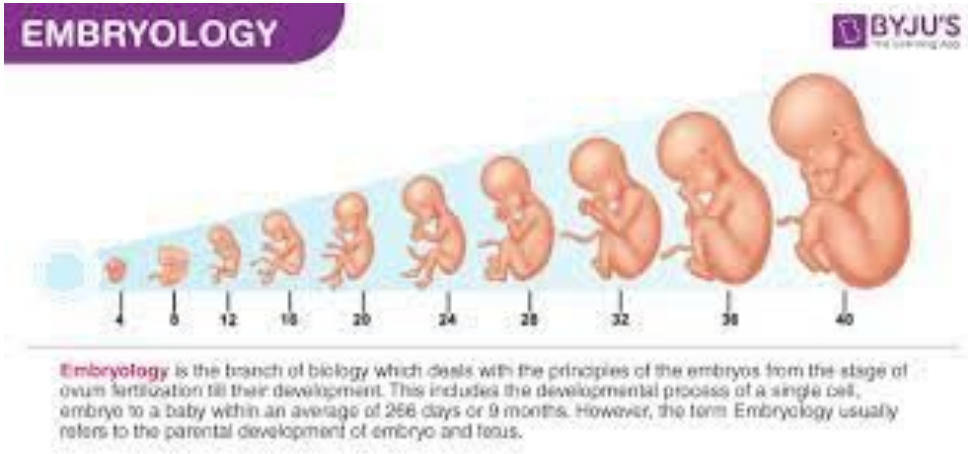
- This architecture aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. It relies entirely on **self-attention** to compute representations of its input and output without using sequence-aligned RNNs or convolution.
- Self-attention is a rich representation which captures the interdependencies between the words compared to single word embeddings. Self-attention adheres to long-term dependencies.
- Three types of attention:

1. Within encoder
2. Encoder - decoder
3. Within decoder

Always Use Pre-trained Model for Transformer (Revisit)

```
transformer_name = "bert-base-uncased"
tokenizer = transformers.AutoTokenizer.from_pretrained(transformer_name)
transformer = transformers.AutoModel.from_pretrained(transformer_name)
```

- It is very expensive to train a Transformer model from scratch.
- Use **transformer embryos** (**pre-trained models** with its **tokenizer**).
- Then, re-train the embryos with your data.
- According to one of the FYP student (Edwin Cheung)
  - Training a BERT model with 3 billions words took **8 days with TPU v3**.
  - Re-training a BERT model with 3 billions words took **2-3 hours with TPU v3**.



Example: TinyBERT Model and Tokenizer

```
model_name = "huawei-noah/TinyBERT_General_4L_312D"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load model
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
).to(device)

```

Code:
https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=9gqrfGY6crr-

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at
huawei-noah/TinyBERT_General_4L_312D and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and
inference.
Model parameters: 14,350,874 (~21% of TinyBERT)

Epoch 1/3
model.safetensors: 100%
 62.7M/62.7M [00:00<00:00, 73.2MB/s]
Batch 0, Loss: 0.6941
Batch 200, Loss: 0.5726
Batch 400, Loss: 0.6420
Batch 600, Loss: 0.3743
Batch 800, Loss: 0.2755
Batch 1000, Loss: 0.2464
Batch 1200, Loss: 0.3941
Batch 1400, Loss: 0.7729
Batch 1600, Loss: 0.2246
Batch 1800, Loss: 0.5349
Batch 2000, Loss: 0.2763
Batch 2200, Loss: 0.2697
Batch 2400, Loss: 0.2823
Batch 2600, Loss: 0.5320
Batch 2800, Loss: 0.1846
Batch 3000, Loss: 0.1035
Train Loss: 0.4241, Train Acc: 0.8048
Test Acc: 0.8343
New best model saved!
```

```
Epoch 2/3
Batch 0, Loss: 0.4508
Batch 200, Loss: 0.6461
Batch 400, Loss: 0.0871
Batch 600, Loss: 0.4221
Batch 800, Loss: 0.2527
Batch 1000, Loss: 0.1524
Batch 1200, Loss: 0.2381
Batch 1400, Loss: 0.2461
Batch 1600, Loss: 0.2131
Batch 1800, Loss: 0.3346
Batch 2000, Loss: 0.1715
Batch 2200, Loss: 0.1069
Batch 2400, Loss: 0.1748
Batch 2600, Loss: 0.3389
Batch 2800, Loss: 0.6598
Batch 3000, Loss: 0.2847
Train Loss: 0.3305, Train Acc: 0.8565
Test Acc: 0.8566
New best model saved!

Epoch 3/3
Batch 0, Loss: 0.1780
Batch 200, Loss: 0.1469
Batch 400, Loss: 0.3093
Batch 600, Loss: 0.4237
Batch 800, Loss: 0.5701
Batch 1000, Loss: 0.2219
```

```
Batch 1200, Loss: 0.0302
Batch 1400, Loss: 0.2444
Batch 1600, Loss: 0.0573
Batch 1800, Loss: 0.2814
Batch 2000, Loss: 0.0465
Batch 2200, Loss: 0.7756
Batch 2400, Loss: 0.0241
Batch 2600, Loss: 0.2938
Batch 2800, Loss: 0.1084
Batch 3000, Loss: 0.4020
Train Loss: 0.2824, Train Acc: 0.8808
Test Acc: 0.8558

Best Test Accuracy: 0.8566
Final Test Accuracy: 0.8566
```

Summary for Transformer Model

1. Eliminating sequential processing through self-attention
2. Enabling full-context understanding from both directions simultaneously
3. Providing massively parallel processing for efficient training
4. Capturing complex linguistic phenomena through multi-head attention
5. Establishing the foundation for all modern LLMs (BERT, GPT, T5, LLaMa, DeepSeek, Gemini, … etc.)
6. However, fine-tuning a transformer model for real world applications and general purposes totally depends on the amount of the capital

Some Available Models for Semantic Analysis

Model Name	Access String	Brief Description	Parameter	Size
ALL-MPNET	`sentence-transformers/all-mpnet-base-v2`	For sentence embeddings and semantic similarity	109M	420 MB
All MiniLM L6 V2	`sentence-transformers/all-MiniLM-L6-v2`	Fast and efficient sentence embeddings	22.7M	80 MB
Universal Sentence Encoder (USE)	`https://tfhub.dev/google/universal-sentence-encoder/4`	Google's general-purpose sentence encoder	~137M	525 MB
RoBERTa	`roberta-large-mnli`	RoBERTa large fine-tuned on MNLI corpus for textual entailment	355M	1.3 GB

Some Available Models for Semantic Analysis

Model Name	Access String	Brief Description	Parameter	Size
BERT-base	`textattack/bert-base-uncased-MNLI`	Original BERT base fine-tuned on MNLI	110M	420 MB
DistilBERT	`distilbert-base-uncased`	Distilled version of BERT	66M	250 MB
RoBERTa	`roberta-large`	RoBERTa large model for high-performance tasks	355M	1.3 GB

Tasks Assessing Language Model

General Language Understanding Evaluation (GLUE)

- Benchmark of nine tasks used in NLP

1. **Corpus of Linguistic Acceptability (CoLA)**
    - Determine if a sentence is grammatically correct
    - Type: Single Sentence
    - Metric: Matthews Correlation
2. **Stanford Sentiment Treebank (SST-2)**
    - Movie review sentiment classification
    - Type: Single Sentence
    - Metric: Accuracy
3. **Microsoft Research Paraphrase Corpus (MRPC)**
    - Identify if two sentences are paraphrases
    - Type: Similarity/Paraphrase
    - Metric: F1/Accuracy

General Language Understanding Evaluation (GLUE)

- Benchmark of nine tasks used in NLP

4. **Quora Question Pairs (QQP)**
    - Rate similarity of two sentences (1-5)
    - Type: Similarity
    - Metric: Pearson/Spearman Correlation
5. **Semantic Textual Similarity Benchmark (STS-B)**
    - Identify if two questions are duplicates
    - Type: Similarity/Paraphrase
    - Metric: F1/Accuracy
6. **Genre Natural Language Inference (MNLI)**
    - Determine if a hypothesis is true given a premise
    - Type: Inference
    - Metric: Accuracy

General Language Understanding Evaluation (GLUE)

- Benchmark of nine tasks used in NLP

7. **Question-answering NLI (QNLI)**
    - Determine if a sentence contains the answer to a question
    - Type: Inference
    - Metric: Accuracy
8. **Recognizing Textual Entailment (RTE)**
    - Binary version of NLI
    - Type: Inference
    - Metric: Accuracy
9. **Winograd NLI (WNLI)**
    - Coreference resolution through NLI
    - Type: Inference
    - Metric: Accuracy

What GLUE assess?

1. **Sentiment Analysis**: Assessing the sentiment of a given text, such as determining whether a customer review is positive or negative.
2. **Text Classification**: Categorizing text into predefined classes or categories based on its content.
3. **Named Entity Recognition**: Identifying and classifying named entities in text, such as person names, organizations, and locations.
4. **Text Similarity**: Measuring the similarity between two pieces of text, which has applications in information retrieval and recommendation systems.
5. **Question Answering**: Automatically finding relevant answers to user questions based on a given context or a set of documents.

Reference: https://h2o.ai/wiki/glue/

Diverse Linguistic Phenomena

- GLUE covers multiple aspects of language understanding:
- **Grammar**: CoLA
- **Sentiment**: SST-2
- **Paraphrase/Semantic Similarity**: MRPC, STS-B, QQP
- **Logical Inference**: MNLI, QNLI, RTE, WNLI

Unified Evaluation Framework

- **Single-number metric**: Average score across all tasks
- **Standardized splits**: Consistent train/validation/test sets
- **Leaderboard**: Public ranking of model performance
- **Test server**: Prevents overfitting to test set

BERT and its Family in GLUE scores

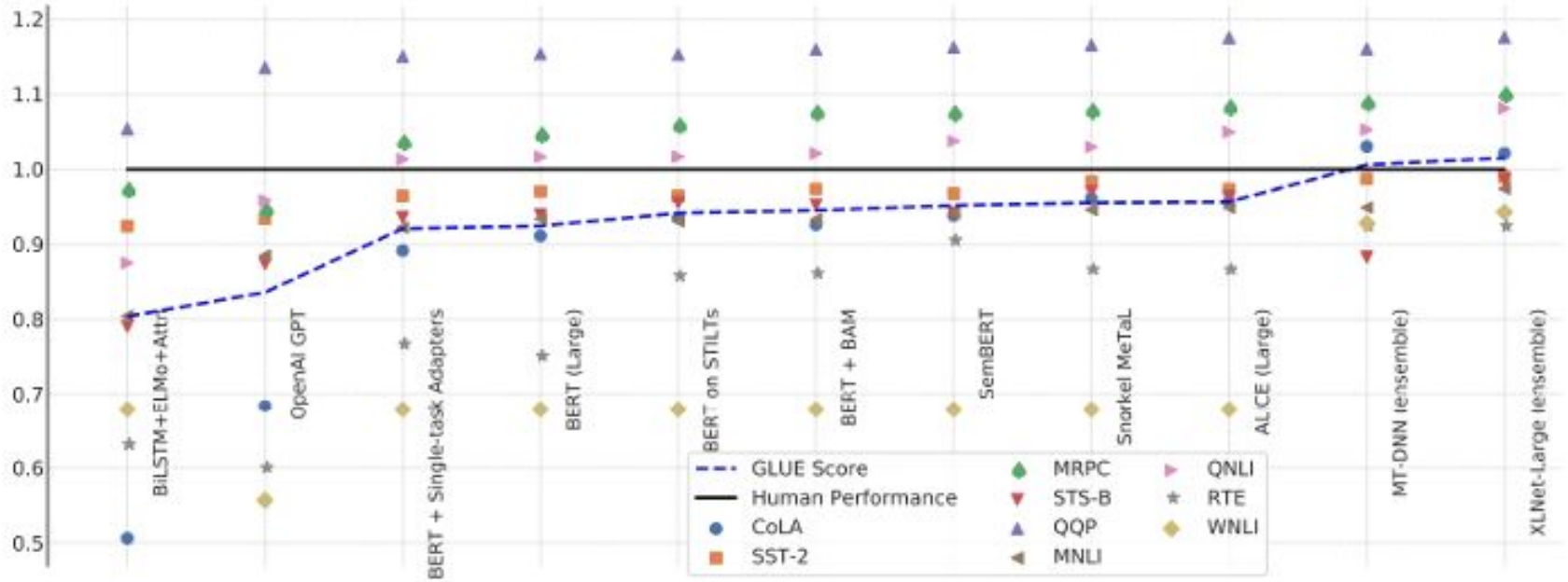


Image credit: https://arxiv.org/abs/1905.00537
To know more about GLUE: https://github.com/nyu-mll/jiant
Code: https://colab.research.google.com/drive/1xxApuWQR_U3naCJ0qj2BcgdHk0MSkIn1#scrollTo=G8N9sqCIRYk6

SQuAD Tasks - SQuAD v1.1

- **Task**: Extract answer span from a context paragraph given a question
- Example:
  - **Context**: “The Amazon rainforest is the largest tropical rainforest in the world, located in South America.”
  - **Question**: “Where is the Amazon rainforest located?”
  - **Answer**: “South America”
- **Significance**: Tests reading comprehension and exact answer extraction
- **Metrics**: Exact Match (EM) and F1 Score

SQuAD Tasks - SQuAD v2.0

- **Task**: Extended version that includes unanswerable questions
- Example:
  - **Context**: “The Amazon rainforest is the largest tropical rainforest in the world.”
  - **Question**: “How tall is the Eiffel Tower?”
  - **Answer**: Unanswerable (not in context)
- **Significance**: Tests ability to recognize when no answer exists
- **Metrics**: Exact Match (EM) and F1 Score with unanswerable handling

Text Generation

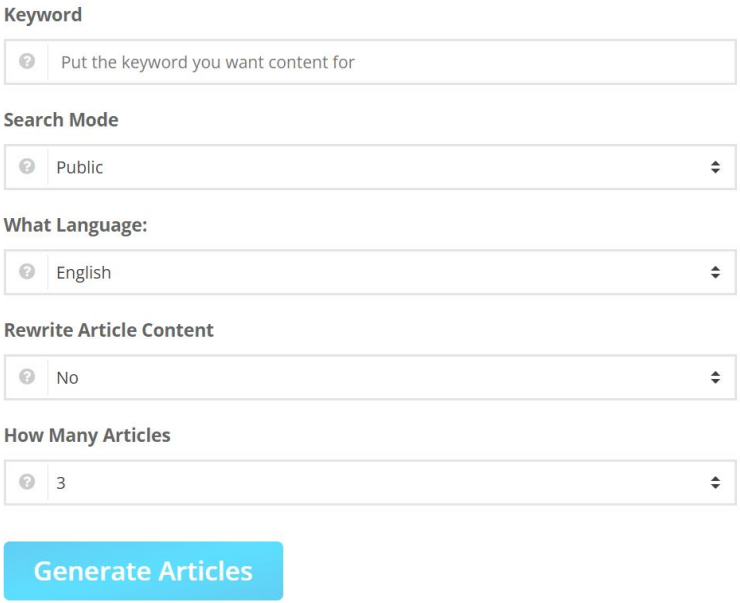
What is Text Generation?

- A Text Generation model, also known as a causal language model, can be trained on code from scratch to **help** the programmers in their **repetitive coding and writing tasks.**
- Text generation is a process where an AI system produces written content, **imitating human language patterns and styles**. The process involves generating **coherent and meaningful text** that resembles natural human communication.
- Mathematically, in a sequence of text, you are going to perform “next word prediction”

$$x_t = f(x_0 \ldots x_{t-1}, c)$$

Text Generation is a more challenging problem than NLP

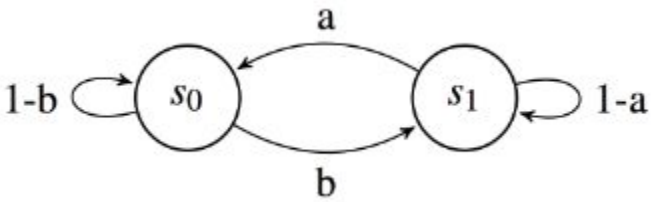
- The content should be coherent and consistent (tone and style).
- The content generated is expected to be different every time.
- The input is usually smaller than the output.



Reference:
https://articlegenerator.org/

Markov Chain Approach (Non deep learning)

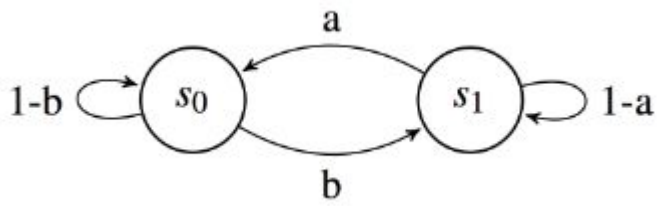
- Basic Markov Chain Model
- **State Space**: $S = \{S_0, S_1, \ldots, S_{k-1}\}$: the set of states that the system is in
- **Transition**:
  - $P(X_n = x_n \mid X_1 = x_1, X_2 = x_2, \ldots, X_{n-1} = x_{n-1}) = P(X_n = X_n \mid X_{n-1} = x_{n-1})$
  - We have transition matrix P such that the i-th row, j-th entry equals $P(X_{n+1} = j \mid X_n = i)$
- **Initial Probability Distribution**: $\pi_0 = [\pi_0(s_0), \pi_0(s_1), \ldots, \pi_0(s_{n-1})]$
  - $\pi_n = \pi_{n-1} P$



Code: https://www.kaggle.com/code/nulldata/meaningful-random-headlines-by-markov-chain/notebook

Steady State Behaviour of Markov Model

- Example:
- State: $\{S_0, S_1\}$
- Transition matrix: $\begin{bmatrix} 1-b & b \\ a & 1-a \end{bmatrix}$

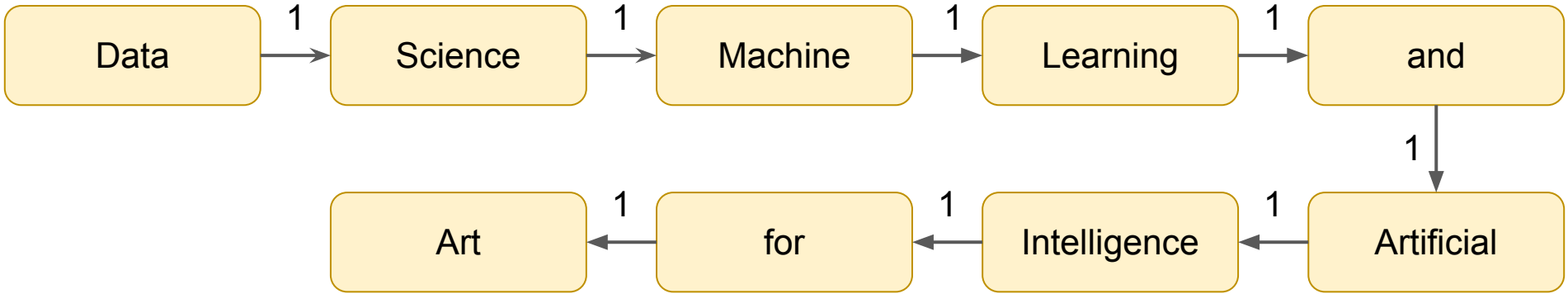


- We can write the relations as: $[p_{0,t+1}\ p_{1,t+1}] = \begin{bmatrix} 1-b & b \\ a & 1-a \end{bmatrix} \begin{bmatrix} p_{0,t} \\ p_{1,t} \end{bmatrix}$

- When the matrix P is Irreducible and Aperiodic, there will always be a steady state in the Markov model (converge).
  - Irreducible matrix: The matrix is not similar via a permutation to a block upper triangular matrix (that has more than one block of positive size).
  - Aperiodic matrix: Every state in the matrix has period 1.

Visualization for Markov Model of Article Title

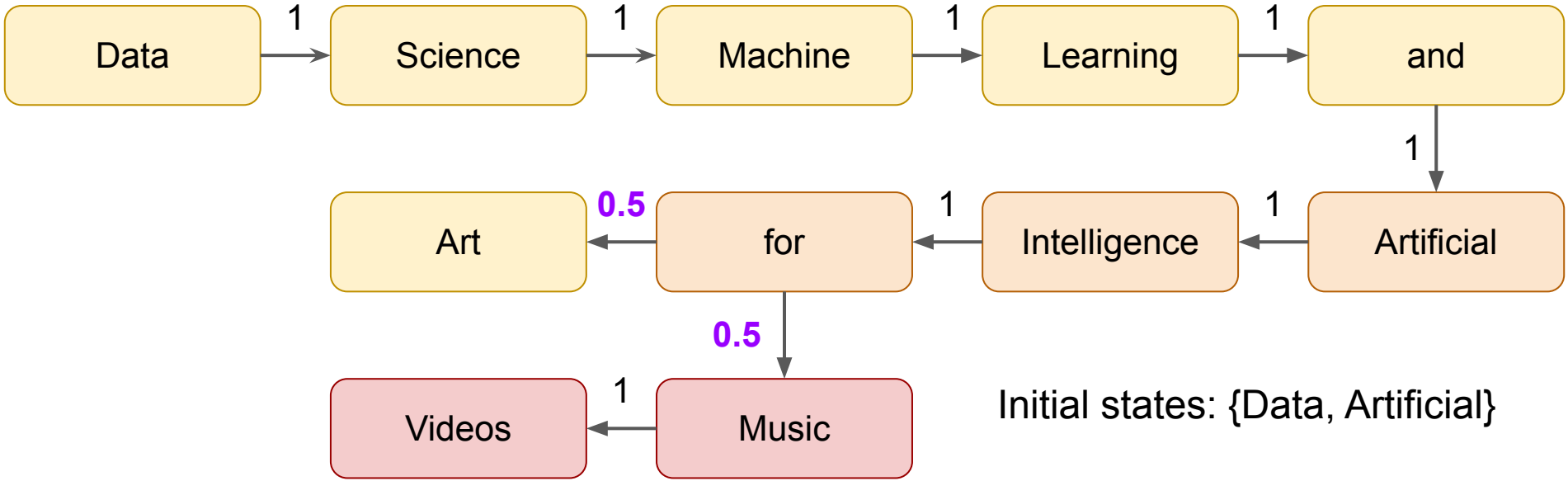
1. Data Science, Machine Learning and Artificial Intelligence for Art
2. Artificial Intelligence for Music Videos



Initial states: {Data}

Visualization for Markov Model of Article Title

1. Data Science, Machine Learning and Artificial Intelligence for Art
2. Artificial Intelligence for Music Videos



Example: Use Markovify Package for Text Generation

1. Load the natural language datasets
2. Separate positive and negative reviews
3. (Optional) Filter the unneeded spaces, indentations, punctuations, chapter headings
4. Fit the document with the Markovify model
5. Generate some text based on the model

Example: Use Markovify Package for Text

```
Generating Positive Reviews:
Positive Review 1: It's got a jazzy, hip, 60s feel to it (lots of color and 70's nostalgia),
Wonderland is dark.
Positive Review 2: To be honest my wife didn't really go with the flow.
Positive Review 3: In any case, the director shows great eye for detail, like the attention
paid in a novel, where the drama rests entirely on the story more and story is phenominal.

Generating Negative Reviews:
Negative Review 1: The dead man's roommate finds himself embroiled in a deadly new drug that
alters a person's DNA and gives them the same bad dream about the Witch and her spiders.
Negative Review 2: And guess what, they really would like to see more new of TMNT series but
just as the band think things couldn't get any worse it surprises you in the very beginning
was tortured in so many ways, but 'Urf is meant to be funny.
Negative Review 3: In spite of sterling work by the supporting actors, and an intelligent
script I have to say if you sat through this thing thinking that with such a ROTTEN movie.
```

Code:
https://colab.research.google.com/drive/1VEJ6zZx6IRKaauJgimR1KO0VPDjJKIKj#scrollTo=UMTHpN-f8xPG

Discussions

1. Markov Chain model does not require any deep learning algorithms
    - Light on memory, fast to execute
2. However, generated content usually mixes different writing style, tone
3. Markov Chain model is unable to predict previous word
4. Markov Chain model cannot handle Polysemy and Homonyms

More Online Examples on Markovify

1. Online tutorial for Markovify:
   https://www.youtube.com/watch?v=9TsuQz9lXis&ab_channel=zachwhalen
2. A Colab demonstration for Markovify:
   https://colab.research.google.com/drive/1szxfgCr-71Qudoh_erKEESSLOhU9Fhh7?usp=sharing
3. A Github demonstration for Markovify:
   https://github.com/NajiAboo/markovify_text_generation
4. Github for Markovify:
   https://github.com/jsvine/markovify

Generative Pre-trained Transformers (GPT) Model

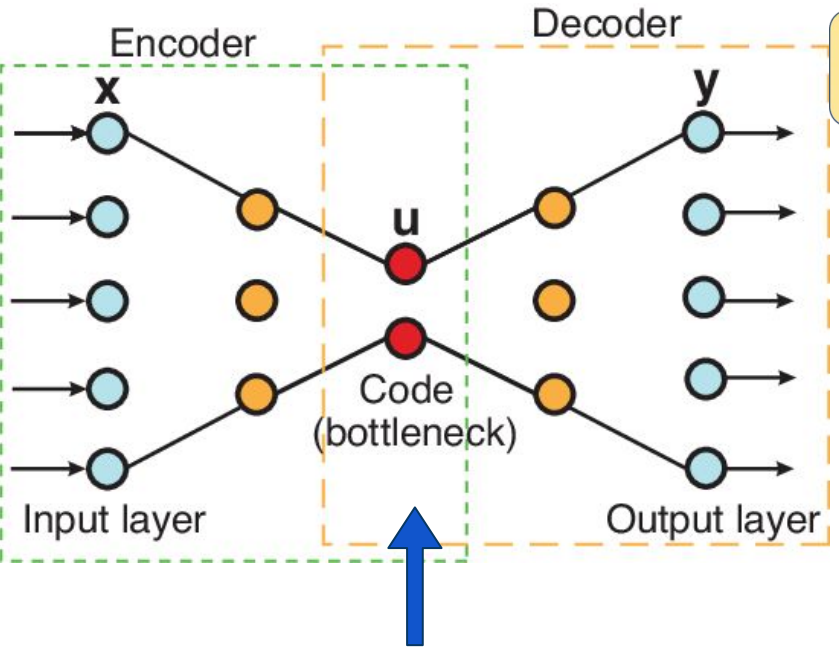
Generative Pre-trained Transformers (GPT) Approach

- GPT model (decoder-based transformers) is used to generate human-like text, such as responses to questions and prompts. It is hard to distinguish the generated text is written by human or not.
- GPT is also known as a Large Language Model (LLM).
- Applications include (i) chatbot and customer service, (ii) content creation, (iii) virtual assistants, (iv) language translation, (v) article summary, (vi) grammar correction, (vii) programming, … etc.

ChatGPT

Generative Pre-trained Transformers (GPT) Approach

- Recall in word embedding, we can employ encoders or transforms for help.



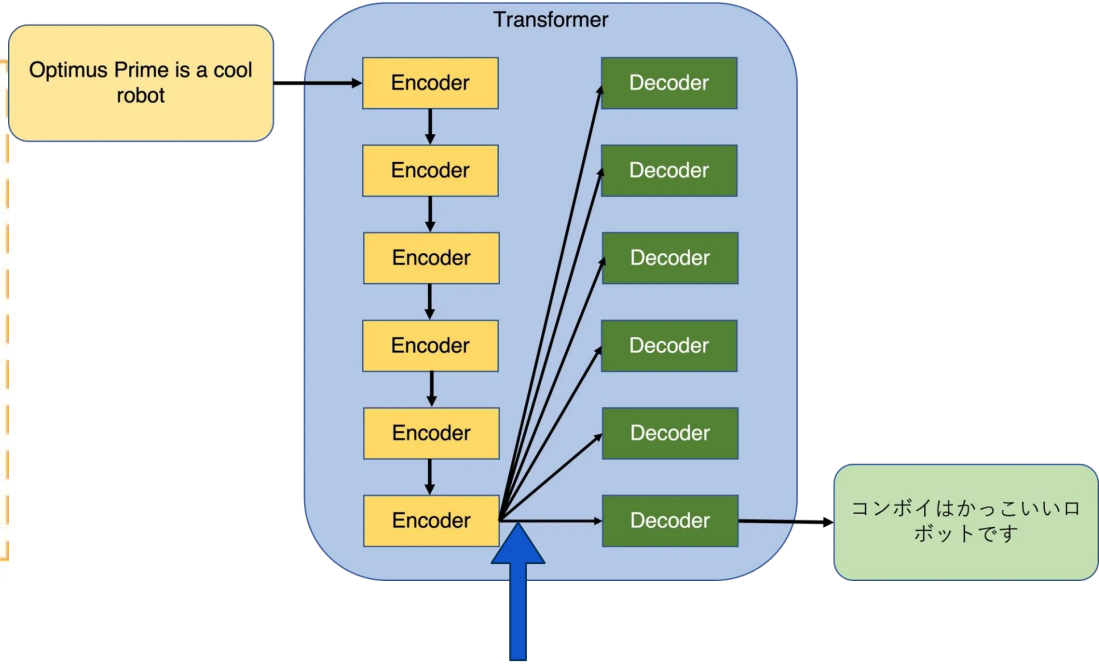
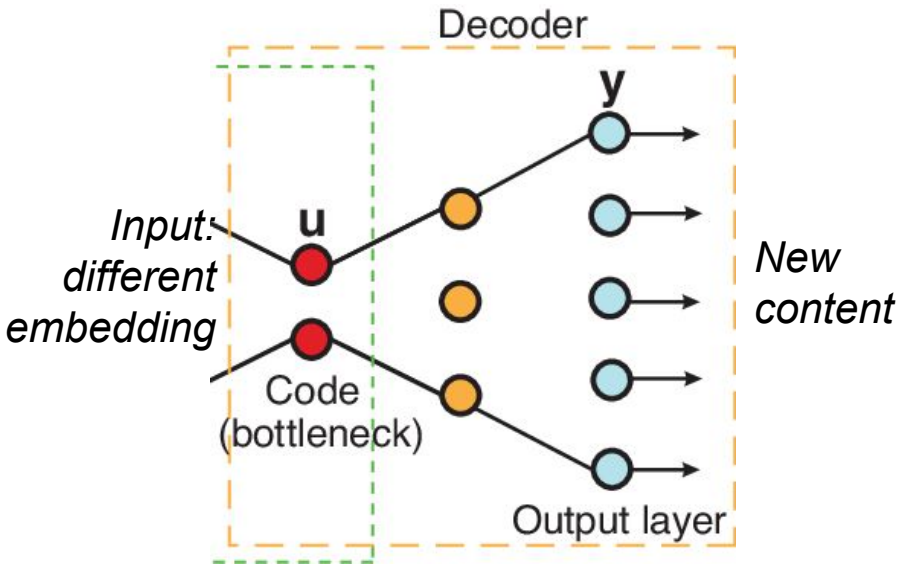


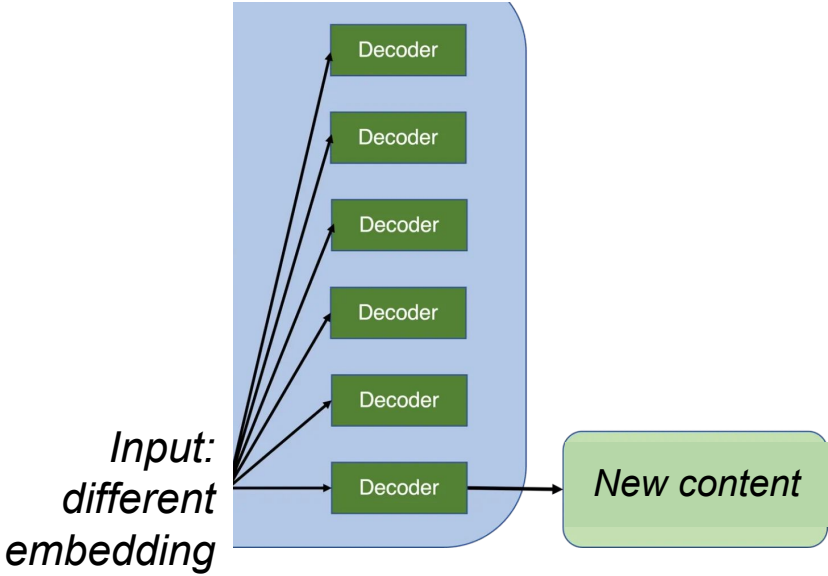
Image credit: ResearchGate, Domino Data Lab

Are there any applications for decoders?

Generative Pre-trained Transformers (GPT) Approach

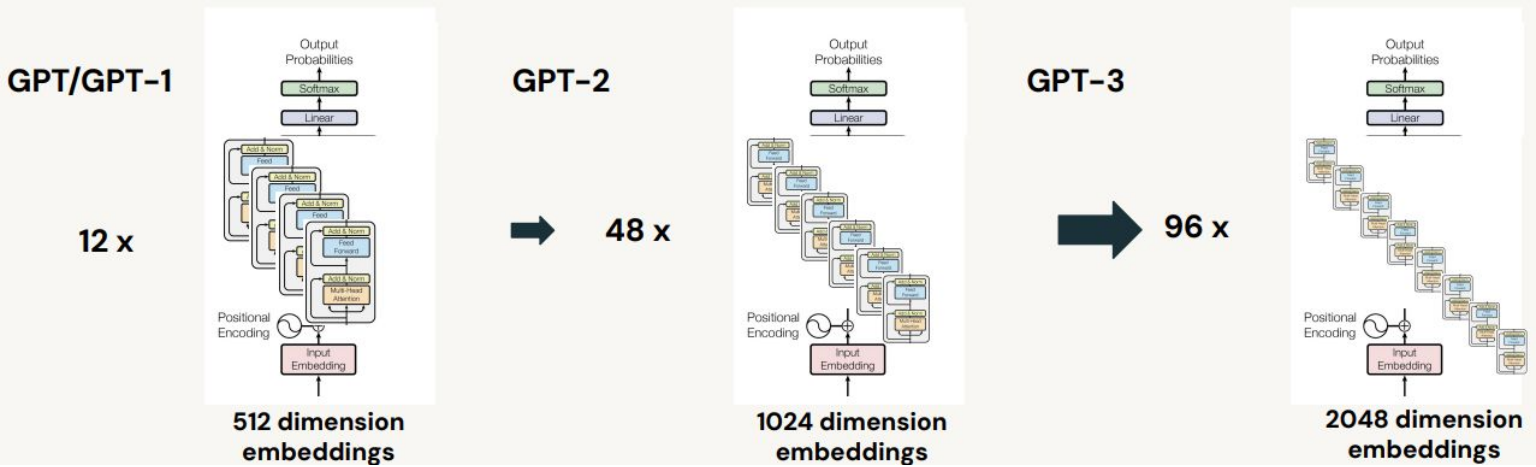
- Inspiration: Use the decoders / generative pre-trained transformers to generate new natural language message.





Generative Pre-trained Transformers (GPT)

- The first GPT model, introduced in 2018 was just the decoder part of the original transformer.
- GPT-2/-3/-4 have mostly just been larger versions. With the key differences coming from training data and training processes.



Online lecture: https://www.youtube.com/watch?v=yMYeNc2fktA&ab_channel=Databricks
Online materials: https://github.com/databricks-academy/llm-foundation-models

Generational Improvements in GPT

1. **GPT (2018):**
   - It was pre-trained on the **BooksCorpus** dataset and contained 117 million parameters. GPT was an autoregressive model based on the Transformer architecture and employed a unidirectional language modeling objective.
2. **GPT-2 (2019):**
   - pre-trained on a much larger dataset called **WebText**, which contained text from web pages with a total of 45 terabytes of data. GPT-2 came in four different sizes: 117M (small), 345M (medium), 774M (large), and 1.5B (extra-large) parameters.

Image credit: https://jalammar.github.io/illustrated-gpt2/

Generational Improvements in GPT

3. **GPT-3 (2020):**
   - Pre-trained on the **WebText2** dataset, an even larger portion of the internet, 45 terabytes of text.
   - GPT-3 came with a staggering 175 billion parameters
   - Exceptional **few-shot** and **zero-shot learning** capabilities, allowing it to perform well on various tasks with minimal or no fine-tuning.
4. **GPT-4 (2023):**
   - Pre-trained on the internet data, academic data, prose data, embedded image data and interleaved data.
   - GPT-4 has 1.76 trillion parameters
   - OpenAI stated that GPT-4 is "more reliable, creative, and able to handle much more nuanced instructions than GPT-3.5."

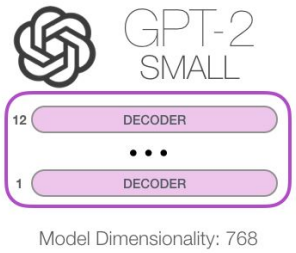
Why GPTs keep getting bigger?

- The main component of each layer: **multi-headed attention block**.
- Like in convolutional layers, the earlier features that are learned might be edges and lines, and the later layers are more complex visual artifacts, attention layers work in a similar fashion:

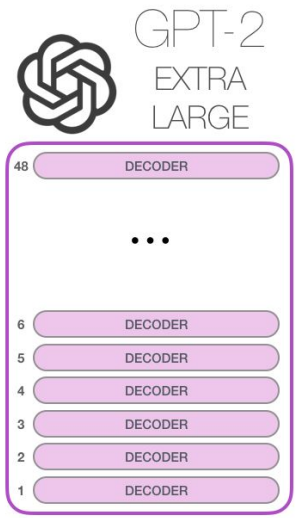
1. **Early Attention**: short-range dependencies, relationships between adjacent or close tokens.
    - Word order, Part-of-Speech, Basic sentence structure.
2. **Middle Attention**: overall context of the input sequence.
    - Semantic information, Meaning, Relationships between phrases, Roles of different words within the sentence.
3. **Late Attention**: integrates the lower layers and generates coherent and contextual outputs.
    - High-level abstractions, Discourse structure, Sentiment, Complex long-range dependencies.

Why are there so many parameters in GPTs?

Parameter	GPT-2 Small	GPT-2 Extra Large	Difference Extra Large vs. Small
Layers	12	48	3x
Model Dimensionality	768	1600	2x
Attention Heads per Layer	12	25	2.5x
Total	**117 Million**	**1.5 Billion**	**12.8x**



Model Dimensionality: 768



Model Dimensionality: 1600

These parameters represents learned knowledge and patterns from vast amounts of data.
On the other hand, the evolution of GPT does not have much changes in the architecture.

Training Data in GPTs

1. **GPT-1: BooksCorpus**
   - Over 7,000 unpublished books spanning various genres.
   - 800 million words, covering a wide range of topics and styles.
2. **GPT-2: WebText**
   - Much larger than BookCorpus
   - A collection of text from web pages, consists of 45 terabytes of data.
   - Filtered out web pages with low quality content
3. **GPT-3: WebText2**
   - Even larger than WebText and more diverse
   - The increased size and diversity of the WebText2 dataset contribute to GPT-3's impressive performance on various NLP tasks.

Comparisons of Different LLMs

	Encoder-only (e.g. BERT)	Decoder-only (e.g. GPT)	Seq-2-Seq (e.g. T5)
Pros	• Pre-trained on a masked language modeling task bidirectional context for a deeper understanding of input sequences. Leads to strong feature extraction capabilities. • Typically several orders of magnitude smaller than decoder or seq-2-seq model.	• Autoregressive nature makes them well-suited for text generation tasks. • Can generate coherent and contextually relevant text.	• Encoder-decoder architecture allows for better handling of complex, structured input-output relationships. • Attention mechanisms help weigh the importance of different parts of the input when generating the output.
Cons	• Not ideal for text generation tasks due to their bidirectional nature. • Can have higher computational costs compared to decoder-only models.	• Only capture unidirectional context (left-to-right), which can limit their contextual understanding. • Autoregressive generation can be slow due to sequential token prediction.	• Can require more training data and computational resources compared to encoder-only or decoder-only models. • Can be more complex to train and fine-tune due to the two-part architecture.

Steps in training a GPT model from scratch

1. **Language models**: Used to create context.
2. **Neural network architecture**: The framework that processes words and generates text with natural-sounding logic.
3. **Generative models**: These are neural networks that can generate new data points from trained data sets. They are useful for various applications such as text generation, image synthesis, speech recognition, and even machine translation.
4. **Epochs**: training iteration, or how many times the same data will be reviewed by the model.
5. **Batch size**: The number of samples used in each iteration.
6. **Self-attention layers**: A process used to identify relationships between different parts of each sentence/paragraph generated by the model.
7. **Dropout layer**: An algorithm designed to help prevent overfitting (when a machine learning model performs too well on specific data sets). This helps ensure that predictions made from new data will be accurate.

Steps in training a GPT model from scratch

8. **Vocabulary size**: Determines how much “lexical space” the system has access to during its calculations.
9. **Disk size required for training input sequences**: How large your drive needs to be for all necessary information related to fit without running out of room while processing through multiple iterations at once.
10. **Hyperparameter optimization techniques**: These need to be applied while the model is being trained so that it can better adapt to different datasets or tasks. This involves setting values like learning rate and momentum decay rates, adjusting dropout layers, and adding regularization components.
11. **Attention score vector**: A numerical representation created by examining similarities between words within sentences/paragraphs being generated so they sound more realistic when read aloud or written down on paper.

Reference: https://botpress.com/blog/how-can-i-train-my-own-gpt-model

Example: DistilGPT2

- Loading the tokenizer and model for DistilGPT2

```
model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    dtype=torch.float32,
    device_map="cpu"
)
```

```
Prompt: Generate a Positive movie review: A thrilling and ...
Generated: Generate a Positive movie review: A thrilling and emotional journey... and the
story of the story is so funny. I am a fan of the film...

Prompt: Generate a Negative movie review: A disappointing ...
Generated: Generate a Negative movie review: A disappointing and poorly paced film... I have
to say it was an excellent film. I could have bought a movie that was...
```

Example: Code Writer by ChatGPT

I apologize for the error in the code that I provided. The `gridify` function is actually part of the `shapely` library, not `geopandas`. Here is the updated code that you can use to create a grid of geodataframe within a polygon:

Copy code

```
import geopandas as gpd
from shapely.geometry import Polygon

# create a polygon
polygon = gpd.GeoDataFrame({'geometry': gpd.GeoSeries([Polygon([(0,0), (0,1),
(1,1), (1,0)])])})

# create a grid of geodataframe within the polygon
from shapely.gridify import gridify
grid = gpd.GeoDataFrame(
    gridify(polygon.geometry[0], cell_size=0.1, geometry='Polygon')
)

# plot the grid
grid.plot()
```

This should create a grid of geodataframe within the polygon, with each cell having a size of 0.1 units. You can adjust the `cell_size` parameter to change the size of the cells in the grid.

Image credit:
Google Image

Example: Article Summary by ChatGPT

In the series finale of The Good Place, titled "Whenever You're Ready," the main characters bid farewell and make their choices about what comes next. Here's what happens to each of them:

1. Jason: After realizing he has experienced everything the universe has to offer, Jason decides it's time to move on. He cooks an inedible plate of spaghetti for Janet and throws a farewell party for himself. However, he lingers outside the exit door until Janet comes back to give her a parting gift. During this time, he learns to appreciate nature and stillness, becoming Jianyu once again.
2. Tahani: Tahani spends her time in the Good Place pursuing personal growth by learning useful skills beyond party planning. When her parents join her in the Good Place and express their love and remorse for how they treated her on Earth, Tahani initially believes it's closure and prepares to leave. However, she realizes that part of her growth was not being obsessed with others' opinions of her. She chooses to stay in the Good Place and trains to become an afterlife architect.
3. Chidi: Chidi, in the most emotional sequence of the finale, struggles to maintain his sense of joy and wonder after reading all the books and sharing special moments with loved ones. Eleanor acknowledges that the rule of Chidis not being allowed to leave because it would make Eleanors sad is selfish. She lets him go, and Chidi leaves behind a hunk-of-the-Bearimy calendar as a token for Eleanor.

Image credit: Google Image

Example: Rewriting the Paragraph by ChatGPT

reword in 150 words I would use the hashtag #PlayfulLearner to describe myself. I choose this because my way of learning consists of experimentation and exploring and playing around with different elements to learn more. Using my previous mistakes, knowledge, and other open resources or help from other people. There are many things that I have learned to do with my own experimentation. A perfect example of this is Thunkable a mobile development platform. I first found out about this in 6th grade and with the help of small youtube tutorials and exploring the platform I was able to make my first few small apps on my own. I continued to learn on my own and make bigger apps using more complex components

If I had to describe myself in one hashtag, it would be #PlayfulLearner. For me, learning is not about following a set of rules or guidelines, but rather about experimenting, exploring, and playing around with different elements to gain knowledge. I often use my previous mistakes and knowledge, as well as open resources and help from others, to expand my skills.

My approach to learning has led me to many discoveries and accomplishments. For instance, when I discovered Thunkable, a mobile development platform, in 6th grade, I was able to make my first few small app[illegible] some small YouTube tutorials and exploring the platform. From th[illegible] on my own and made more

Regenerate response

Image credit: Google Image

Large Language Model Meta AI (LLaMA) Model

Reference: https://www.llama.com/docs/overview/

LLaMA

- LLaMA (Large Language Model Meta AI) is a family of LLMs developed by Meta AI (formerly Facebook).
  - First released in February 2023
  - Version 2 released on July 2023
  - Version 3 released on April 2024
  - Version 4 released on April 2025
- LLaMa models are **free for research and commercial** use.
- If you look for the lightweight versions, you may try LLaMa 3.2 (1B and 3B).
- The largest LLaMa model is LLaMa 4 Maverick, containing 17B active parameters x 128 experts, roughly 400B total parameters.

LLaMa:
https://www.llama.com/llama-downloads/?utm_source=llama-home-hero&utm_medium=llama-referral&utm_campaign=llama-utm&utm_offering=llama-downloads&utm_product=llama

LLaMA Stack

- Llama Stack standardizes the core building blocks simplifying AI app dev

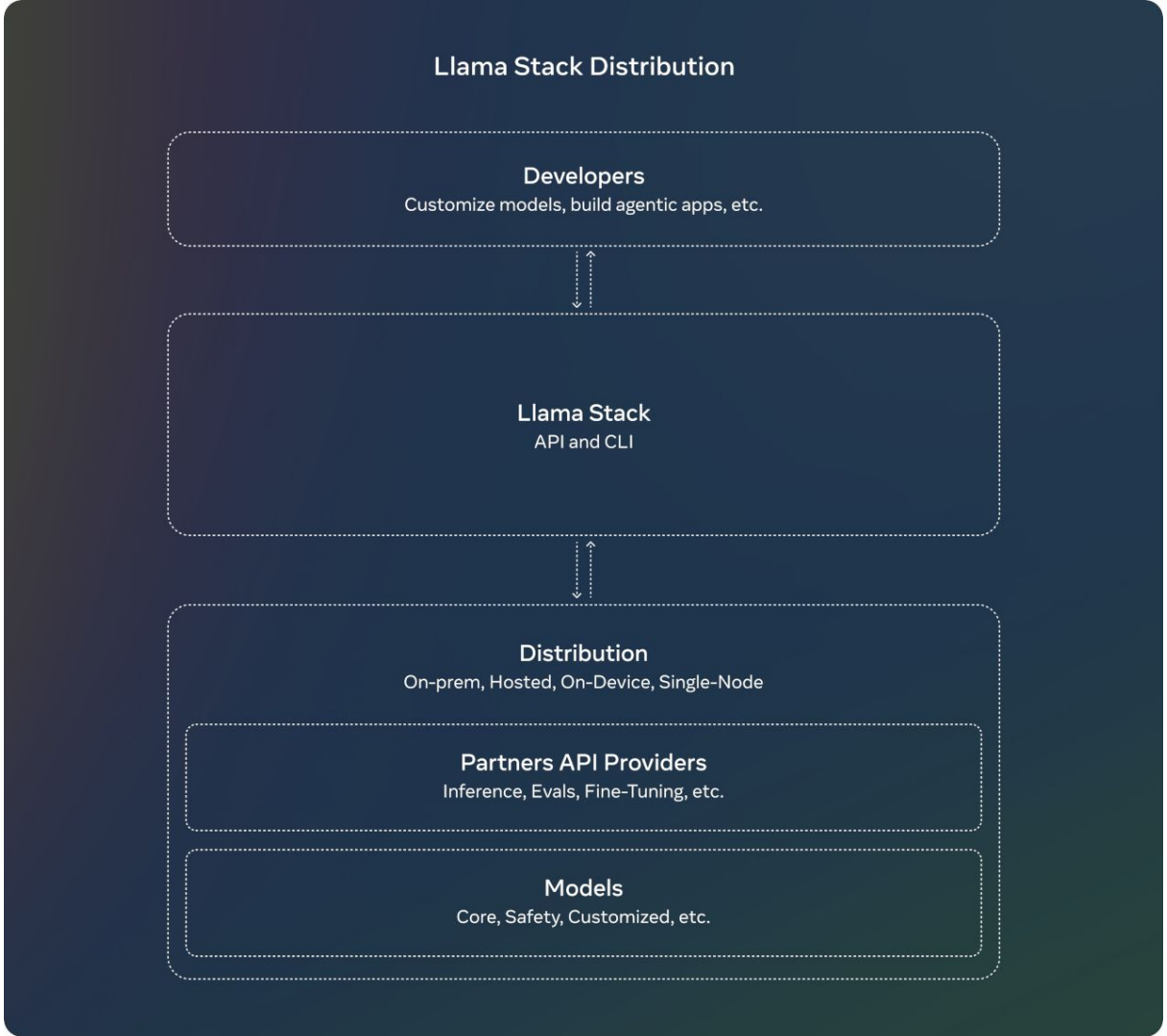
1. **Unified API layer** for Inference, RAG, Agents, Tools, Safety, Evals.
2. **Plugin architecture** to support the rich ecosystem of different API implementations in various environments, including local development, on-premises, cloud, and mobile.
3. **Prepackaged verified distributions** which offer a one-stop solution for developers to get started quickly and reliably in any environment.
4. **Multiple developer interfaces** like CLI and SDKs for Python, Typescript, iOS, and Android.
5. **Standalone applications** as examples for how to build production-grade AI applications with Llama Stack.

Reference: https://github.com/llamastack/llama-stack

Image credit: https://thenewstack.io/llama-stack-released-to-help-developers-build-agentic-apps/

Agentic AI refers to artificial intelligence systems that can autonomously perceive, plan, and execute complex tasks to achieve specific goals, rather than just responding to individual prompts or performing single-step tasks.

Image credit: https://thenewstack.io/llama-stack-released-to-help-developers-build-agentic-apps/



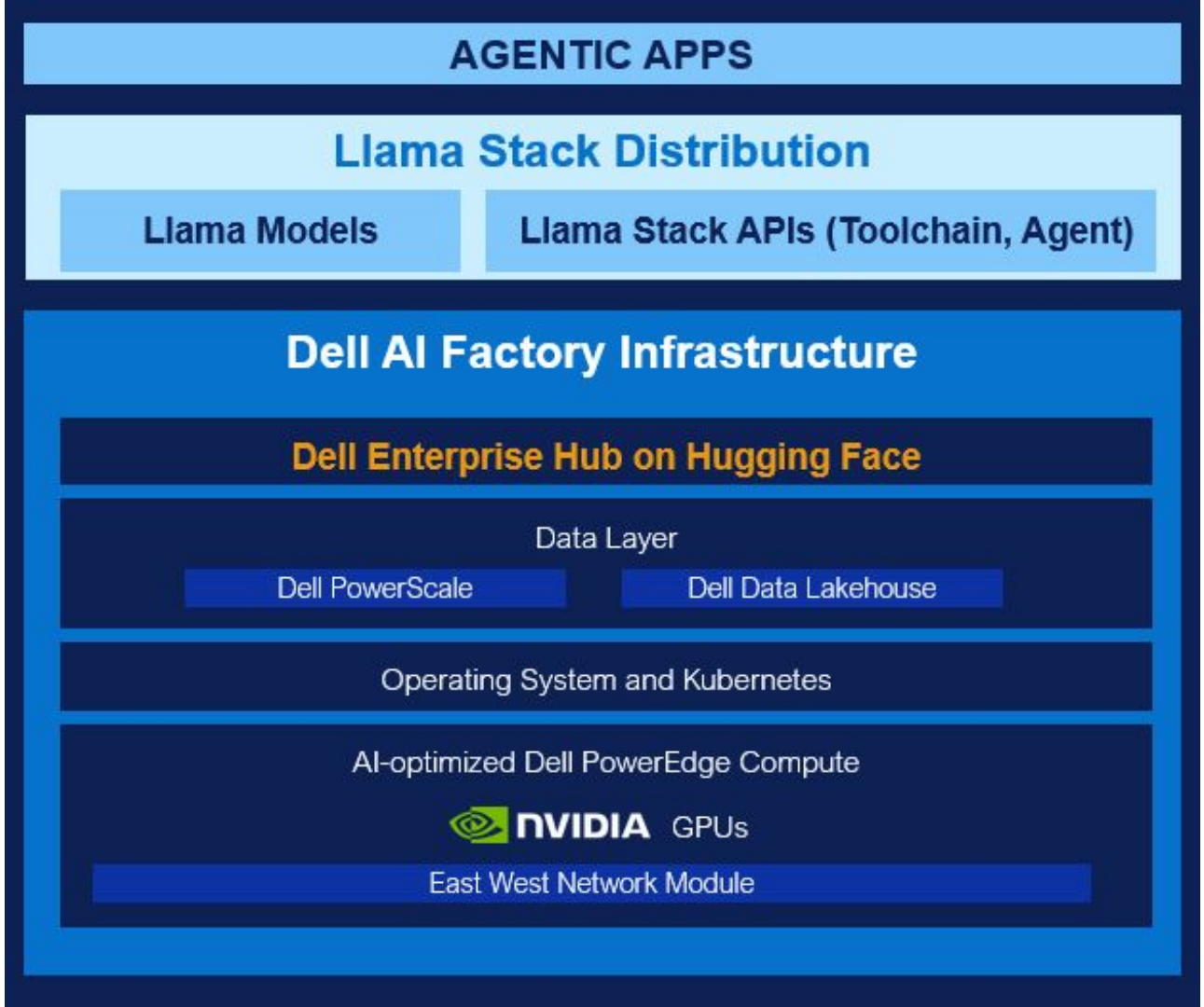


Image credit: https://thenewstack.io/llama-stack-released-to-help-developers-build-agentic-apps/

Agentic System AP

1. **Central Importance**: The "agentic system" API is the most significant component due to:
   - High excitement around AI agents within the AI developer community
   - Positioned at the top of the stack for both Meta and Dell
   - Expected to be regularly used by AI engineers
2. **Framework Identity**:
   - Early commentators identified the Llama Stack (proposed July RFC) as Meta's "agentic framework"
   - Meta uses the term "agentic applications" to describe apps built using Llama Stack

Reference: https://ai.meta.com/blog/5-steps-to-getting-started-with-llama-2/

Agentic System AP

3. **Pre-release Examples**:
   - Meta released GitHub repository examples of appl built on Llama Stack
   - Starting with Llama 3.1, developers can build agentic applications capable of:
4. **Core Capabilities**:
   - **Task Decomposition**: Breaking tasks down and performing multistep reasoning
     - **Built-in tools**: Model has inherent knowledge of tools like search or code interpreter
     - **Zero-shot learning**: Model can learn to call previously unseen tools using in-context definitions
   - **Safety**: System-level protection using models like Llama Guard

Summary

- We have gone through two famous problems in NLP, which are (i) **semantic analysis** and (ii) **text generation**.
- The problems can be solved in either (i) non-deep learning approach and (ii) deep learning approach.
- For non-deep learning approach, we covered **Markov model**.
- For deep learning approach, we highlighted **Neural Bag-of-Words (NBOW)**, **Encoder-only Transformer (BERT)**, **Decoder-only Transformer (GPT)**, **Transformer** and **Large Language Model Meta AI (LLaMA)**.
- Huge computation power and memory space are required when working with LLMs, especially transformer related models.